

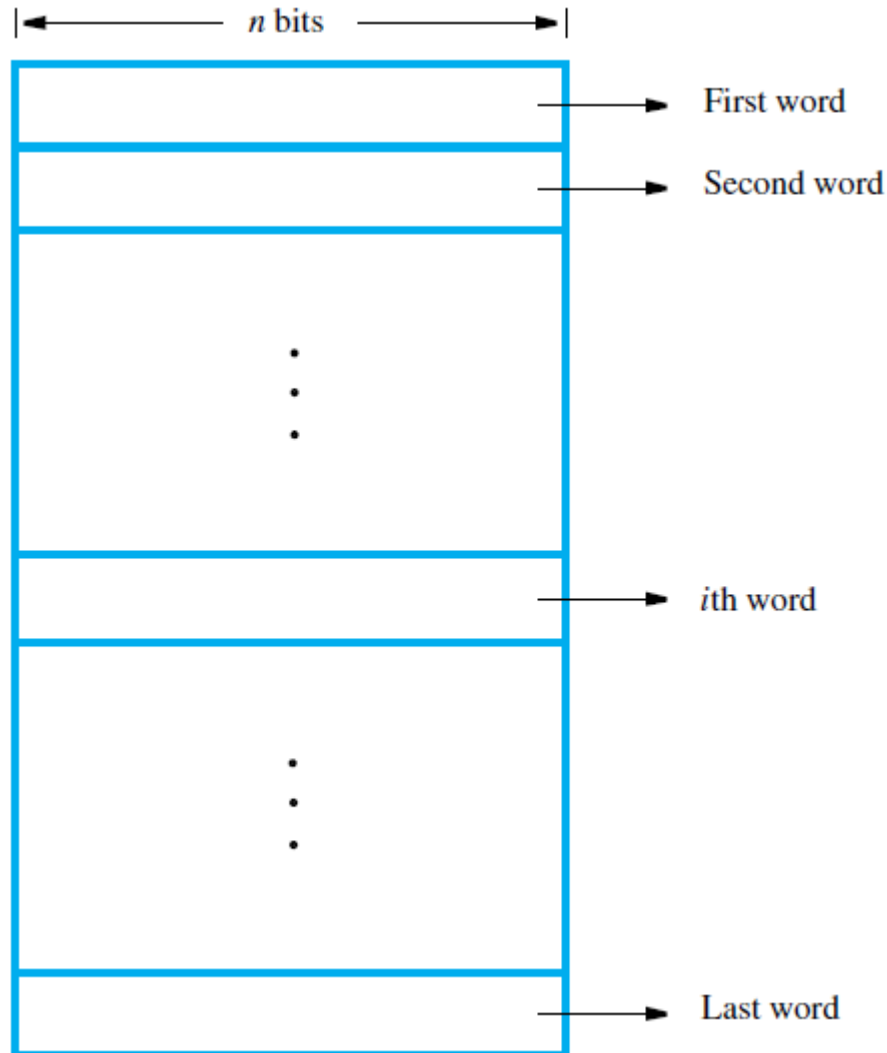
Memory Locations and Addresses

Dr Anjaneyulu Pasala

Memory Locations

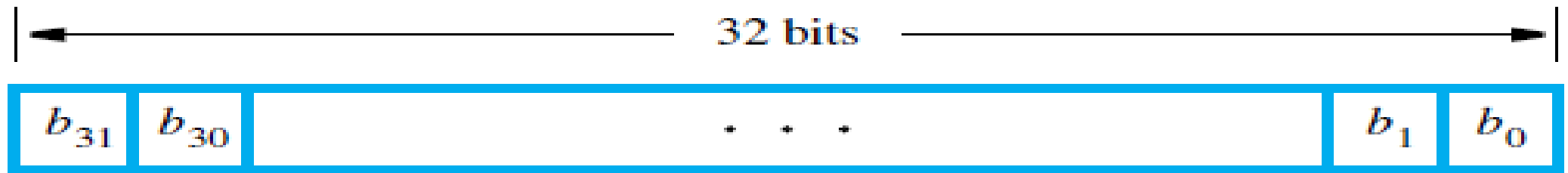
- How is the memory of a computer organized?
- The memory consists of many millions of storage **cells**, each of which can store a **bit** of information having the value 0 or 1.
- The bits are seldom handled individually.
- The usual approach is to deal with them in groups of **fixed size**.
- A group of ***n*** bits can be stored or retrieved in a single, basic operation.
- Each group of ***n*** bits is referred to as a **word** of information, and ***n*** is called the **word length**.
- The memory of a computer can be schematically represented as a collection of words.

Memory Locations



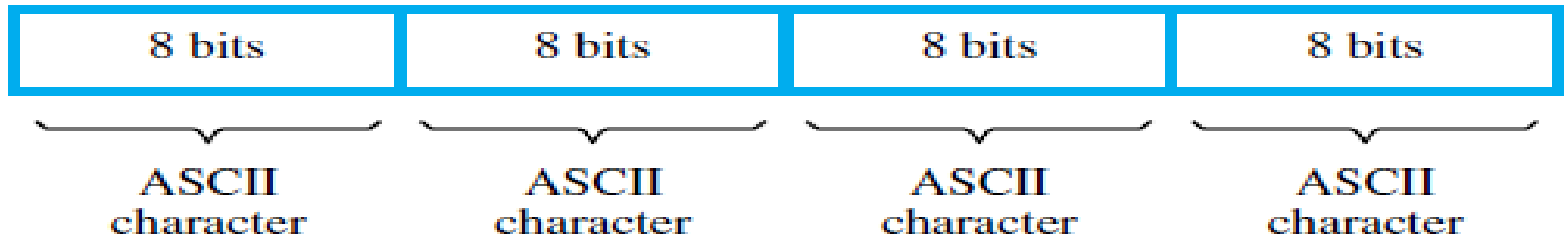
- Modern computers have word lengths that typically range from 32 to 64 bits.
- If the word length of a computer is 32 bits, a single word can store a 32-bit signed number or four ASCII-encoded characters, each occupying 8 bits.
- A unit of 8 bits is called a ***byte***.

Memory Locations



↑ Sign bit: $b_{31} = 0$ for positive numbers
 $b_{31} = 1$ for negative numbers

(a) A signed integer



(b) Four characters

Memory Locations and addresses

- Machine instructions may require one or more words for their representation.
- Accessing the memory to store or retrieve a single item of information, either a word or a byte, requires distinct names or addresses for each location.
- It is customary to use numbers from **0** to $2^k - 1$, for some suitable value of **k**, as the addresses of successive locations in the memory.
- Thus, the memory can have up to 2^k addressable locations.
- The 2^k addresses constitute the address space of the computer.
- An Example: 24-bit address generates an address space of 2^{24} (16,777,216) locations (16M(mega))

Memory Locations and addresses

For example, a 24-bit address generates an address space of 2^{24} (16,777,216) locations. This number is usually written as

16M

(16 mega), where **1M** is the number 2^{20} (1,048,576). A 32-bit address creates an address space of 2^{32} or

4G

(4 giga) locations, where **1G** is 2^{30} .

Byte Addressability

- We have three basic information quantities to deal with: **bit**, **byte**, and **word**.
- A **byte** is always **8** bits
- The **word** length typically ranges from **16** to **64** bits, now 128 bits.
- It is impractical to assign distinct addresses to individual bit locations in the memory. The most practical assignment is to have successive addresses refer to successive byte locations in the memory.
- This is the assignment used in most modern computers. The term **byte-addressable memory** is used for this assignment. Byte locations have addresses **0, 1, 2,**
- Thus, if the word length of the machine is **32** bits, successive words are located at addresses **0, 4, 8, . . .**, with each word consisting of four bytes.
- Word length of 64 bits .. Successive words are located at addresses 0, 8, 16, 32, ...

Big-Endian and Little-Endian Assignments

- There are two ways that byte addresses can be assigned across words.
- When lower byte addresses are used for the **more significant bytes** (the leftmost bytes) of the word it is called **big-endian**.
- where the lower byte addresses are used for the **less significant bytes** (the rightmost bytes) of the word it is called **big-endian**.
- The words “more significant” and “less significant” are used in relation to the weights (powers of 2) assigned to bits when the word represents a number.
- Both little-endian and big-endian assignments are used in commercial machines. Motorola 68K – big-endian, Intel x86 – little-endian.

Big-Endian and Little-Endian Assignments

Word address	Byte address			
0	0	1	2	3
4	4	5	6	7
$2^k - 4$	$2^k - 4$	$2^k - 3$	$2^k - 2$	$2^k - 1$

(a) Big-endian assignment

	Byte address			
0	3	2	1	0
4	7	6	5	4
$2^k - 4$	$2^k - 1$	$2^k - 2$	$2^k - 3$	$2^k - 4$

(b) Little-endian assignment

Word Alignment

- In the case of a 32-bit word length, natural word boundaries occur at addresses **0, 4, 8,** We say that the word locations have aligned addresses if they begin at a byte address that is a multiple of the number of bytes in a word.
- For practical reasons associated with manipulating binary-coded addresses, the number of bytes in a word is a power of 2.
- Hence, if the word length is **16** (2 bytes), aligned words begin at byte addresses **0, 2, 4, . . . ,** and for a word length of **64** (2^3 bytes), aligned words begin at byte addresses **0, 8, 16,**

Addressing Numbers, characters and Character Strings

- Usually, a number occupies one word, hence, can be accessed by specifying its word address
- Individual characters can be accessed by their byte address
- Character strings of variable lengths
 - The beginning of the string is indicated by giving address of the byte containing the first character
 - Successive byte locations contain successive characters of the string.
 - Length of the string is indicated by
 - A special character (control character) at the end of the string as a last character in the string
 - A separate memory location can contain a number indicating the length of the string in bytes

Memory Operations

- Both program instructions and data operands are stored in the memory.
- To execute an instruction, the processor control circuits must cause the word (or words) containing the instruction to be transferred from the memory to the processor.
- Operands and results must also be moved between the memory and the processor.
- Thus, two basic operations involving the memory are needed, namely, **Read** (load, fetch) and **Write** (store)

Memory Operations (Read)

- **The Read operation** transfers a copy of the contents of a specific memory location to the processor. The memory contents remain unchanged.
- To start a Read operation, the processor sends the address of the desired location to the memory and requests that its contents be read.
- The memory reads the data stored at that address and sends them to the processor.

Memory Operations (Write)

- **The Write operation** transfers an item of information from the processor to a specific memory location, **overwriting the former contents** of that location.
- To initiate a Write operation, the processor sends the address of the desired location to the memory, together with the data to be written into that location.
- The memory then uses the address and data to perform the write.

Instructions and Instruction Sequencing

Memory Operations

- The tasks carried out by a computer program consist of a sequence of small steps,
 - Adding two numbers, testing for a particular condition, reading a character from the keyboard, or sending a character to be displayed on a display screen.
- A computer must have **instructions** capable of performing four types of operations:
 - Data transfers between the memory and the processor registers
 - Arithmetic and logic operations on data
 - Program sequencing and control
 - I/O transfers

Register Transfer Notation

- We need to describe the transfer of information from one location in a computer to another.
 - Possible locations that may be involved in such transfers are memory locations, processor registers, or registers in the I/O subsystem.
 - Most of the time, we identify such locations symbolically with convenient names.
- **Example:**
- Names that represent the addresses of memory locations may be **LOC**, **PLACE**, **A**, or **VAR2**
- Predefined names for the processor registers may be **R0** or **R5**.
- Registers in the I/O subsystem may be identified by names such as **DATIN** or **OUTSTATUS**.

Register Transfer Notation

- To describe the transfer of information, the contents of any location are denoted by placing square brackets around its name. Thus, the expression

$$\mathbf{R1} \leftarrow [\mathbf{LOC}]$$

- means that the contents of memory location **LOC** are transferred into processor register **R1**.

Register Transfer Notation

- As another example, consider the operation that adds the contents of registers **R1** and **R2**, and places their sum into register **R3**. This action is indicated as

$$\mathbf{R3} \leftarrow [\mathbf{R1}] + [\mathbf{R2}]$$

- This type of notation is known as **Register Transfer Notation** (RTN).
- Note: The right hand side of an RTN expression always denotes a value, and the left-hand side is the name of a location where the value is to be placed, overwriting the old contents of that location.

Assembly-Language Notation

- We need another type of notation to represent machine instructions and programs. For this, we use assembly language. For example, a generic instruction that causes the transfer described above, from memory location **LOC** to processor register **R1**, is specified by the statement

Move LOC, R1

- The contents of **LOC** are unchanged by the execution of this instruction, but the old contents of register **R1** are overwritten.
- The name Load is appropriate for this instruction, because the contents read from a memory location are loaded into a processor register.

Load LOC, R1

Assembly-Language Notation

- The second example of adding two numbers contained in processor registers **R1** and **R2** and placing their sum in **R3** can be specified by the assembly-language statement

Add R1 , R2 , R3

- In this case, registers **R1** and **R2** hold the source operands, while **R3** is the destination.

Basic Instruction Types

- The fundamental capability of any computers is adding 2 numbers
 - $C = A + B$ A,B and C are called variables
 - $C \leftarrow [A] + [B]$
 - Single machine instruction can be called ***three-address instruction*** can be represented symbolically as **Add A,B,C**
 - General instruction format is
 - Operation Source1, Source2, Destination
 - For a modern processor with a 32-bit instruction address space, a 3-address instruction is too large to fit in one word for a reasonable word length.
 - Hence, we use two-address instructions of the form Add A,B means $B \leftarrow [A] + [B]$
 - Remember contents of location B is lost
 - This problem can be solved by **Move B,C** means $C \leftarrow [B]$
 - Hence $C \leftarrow [A] + [B]$ can be performed as follows
Move B,C
Add A,C
- All instructions use *operation source, destination* format

Basic Instruction Types

- Even 2-address instructions will not fit into one word for usual word lengths and address sizes
- When a second operand is needed, as in the case of an **Add** instruction, it is understood implicitly to be a unique location.
- A process register, called accumulator is used for this purpose.
- These instructions are called one-address instructions.
 - **Add A** means $accumulator = [A] + [Accumulator]$
 - Similarly
 - Load A copies the contents of A into accumulator
 - Store A loads the contents of the accumulator into register A
 - $C \leftarrow [A] + [B]$
 - Load A
 - Add B
 - Store C
 - In one-instruction the operand may be source or destination depends on the type of operation the other operand is accumulator

Basic Instruction Types

- Present generation processors have many general purpose registers, which can perform the role of accumulators
- Let R_i represent a general purpose register. The instructions are:
 - Load A, R_i
 - Store R_i , A
 - Add A, R_i
- Here R_i performs the role of the accumulator
- In many processors, computations are performed directly only on data held in processor registers. Examples: *Add R_i, R_j* and *Add R_i, R_j, R_k*
- Therefore, $C \leftarrow [A] + [B]$

Move A, R_i
Move B, R_j
Add R_i, R_j
Move R_i, C

Or

Move A, R_i
Add B, R_i
Move R_i, C

RISC and CISC Instruction Sets (RISC)

- One of the most important characteristics that distinguish different computers is the nature of their instructions.
- One popular approach is based on the premise that higher performance can be achieved if each instruction occupies exactly one word in memory, and all operands needed to execute a given arithmetic or logic operation specified by an instruction are already in processor registers.
- Such computers are called **Reduced Instruction Set Computers (RISC)**.

RISC and CISC Instruction Sets (CISC)

- An alternative to the RISC approach is to make use of more complex instructions which may span more than one word of memory, and which may specify more complicated operations.
- This approach was prevalent prior to the introduction of the RISC approach in the 1970s.
- Although the use of complex instructions was not originally identified by any particular label, computers based on this idea have been subsequently called **Complex Instruction Set Computers (CISC)**.

Instruction Execution and Straight-Line Sequencing

- The required action can be accomplished by a sequence of simple machine instructions.
- We choose to use registers **R0** to perform the task with three instructions:

```
Move A, R0  
Add B, R0  
Move R0, C
```

Instruction Execution and Straight-Line Sequencing

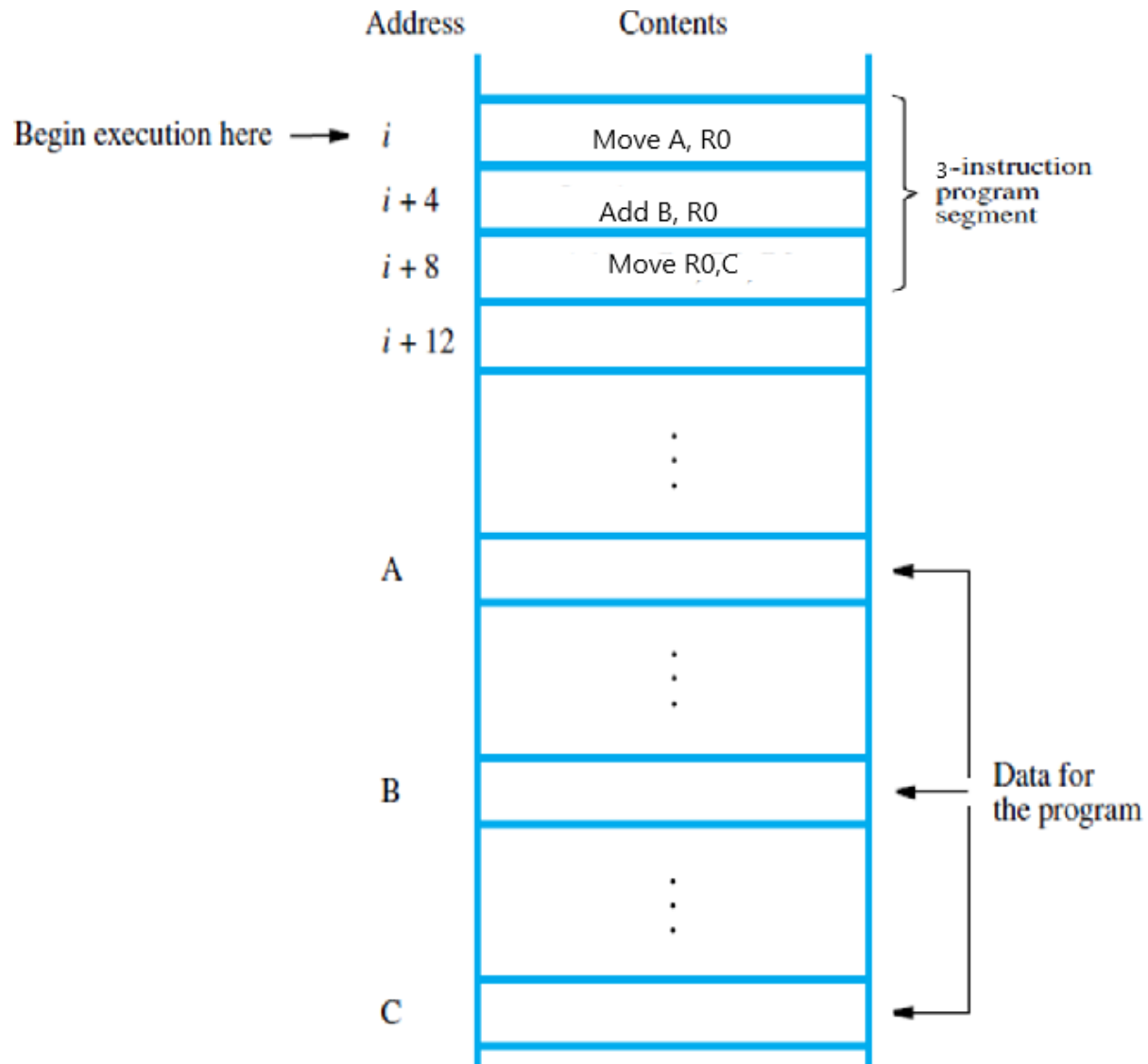


Figure shows a possible program segment for this task as it appears in the memory of a computer.

Instruction Execution and Straight-Line Sequencing

- The processor contains a register called the *program counter* (**PC**), which holds the address of the next instruction to be executed.
- To begin executing a program, the address of its first instruction (i in our example) must be placed into the **PC**.
- Then, the processor control circuits use the information in the **PC** to fetch and execute instructions, one at a time, in the order of increasing addresses. This is called **straight-line sequencing**.
- During the execution of each instruction, the **PC** is incremented by 4 to point to the next instruction. Thus, after the Move instruction at location $i + 8$ is executed, the **PC** contains the value $i + 12$, which is the address of the first instruction of the next program segment.

Instruction Execution and Straight-Line Sequencing

Executing a given instruction is a two-phase procedure.

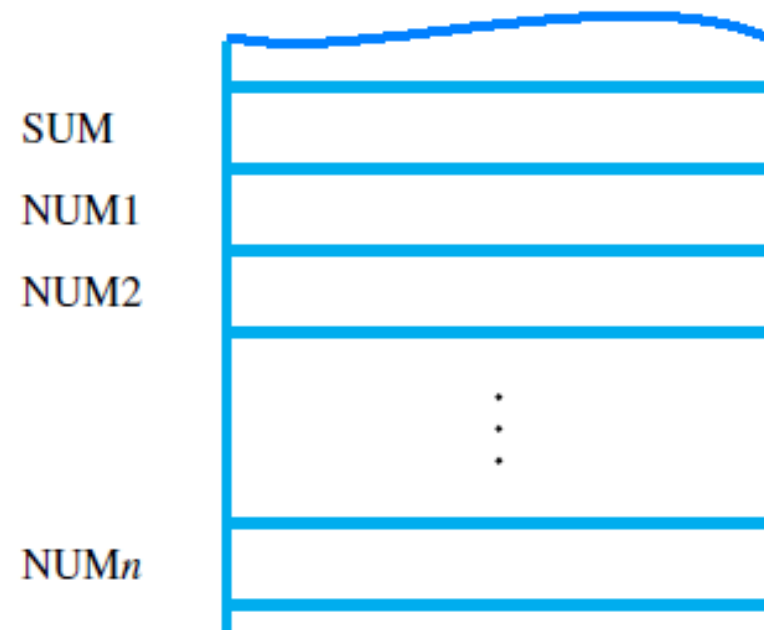
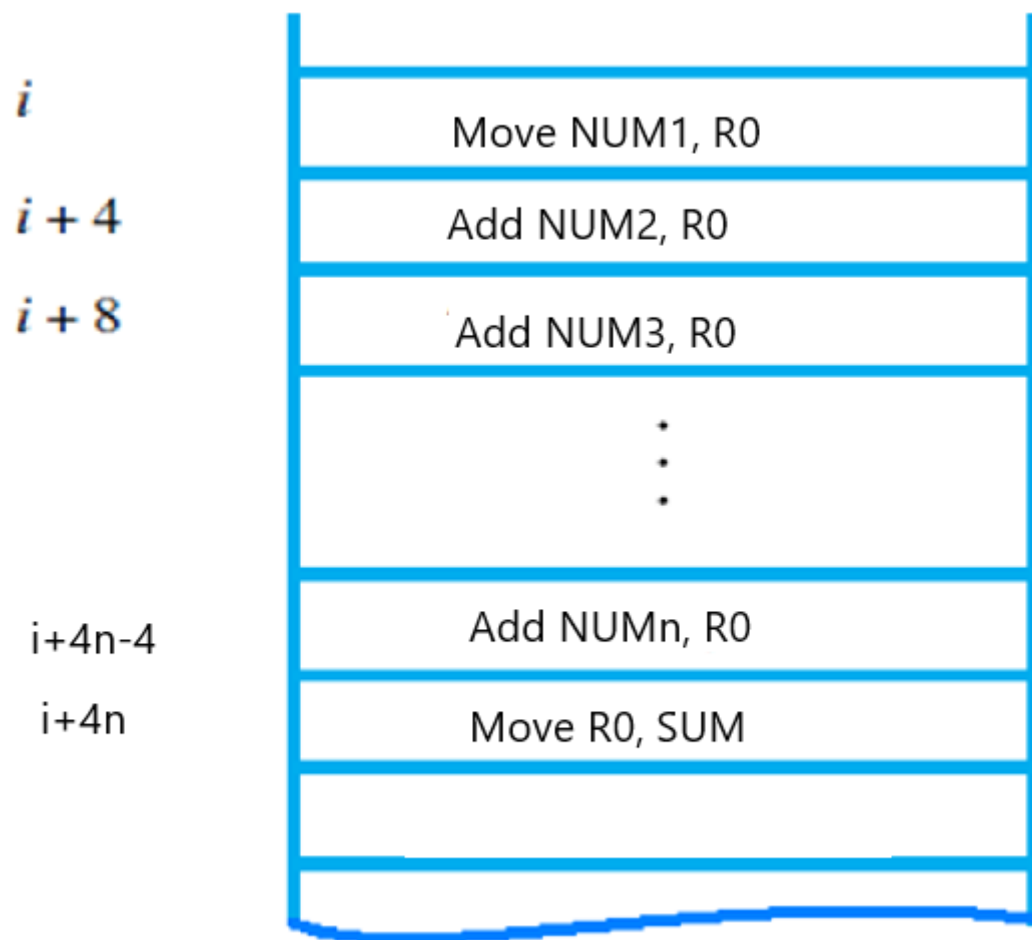
- In the first phase, called ***instruction fetch***, the instruction is fetched from the memory location whose address is in the **PC**. This instruction is placed in the ***instruction register (IR)*** in the processor.
- At the start of the second phase, called ***instruction execute***, the instruction in **IR** is examined to determine which operation is to be performed.
 - The specified operation is then performed by the processor.
 - This involves a small number of steps such as fetching operands from the memory or from processor registers, performing an arithmetic or logic operation, and storing the result in the destination location.
 - At some point during this two-phase procedure, the contents of the **PC** are advanced to point to the next instruction.

Branching

Consider the task of adding a list of n numbers.

- This program is a generalization of the program in previous Figure.
- The addresses of the memory locations containing the n numbers are symbolically given as **NUM1**, **NUM2**, . . . , **NUM n** , and separate **Add** instruction is used to add each number to the contents of register **R0**.
- After all the numbers have been added, the result is placed in memory location **SUM**.

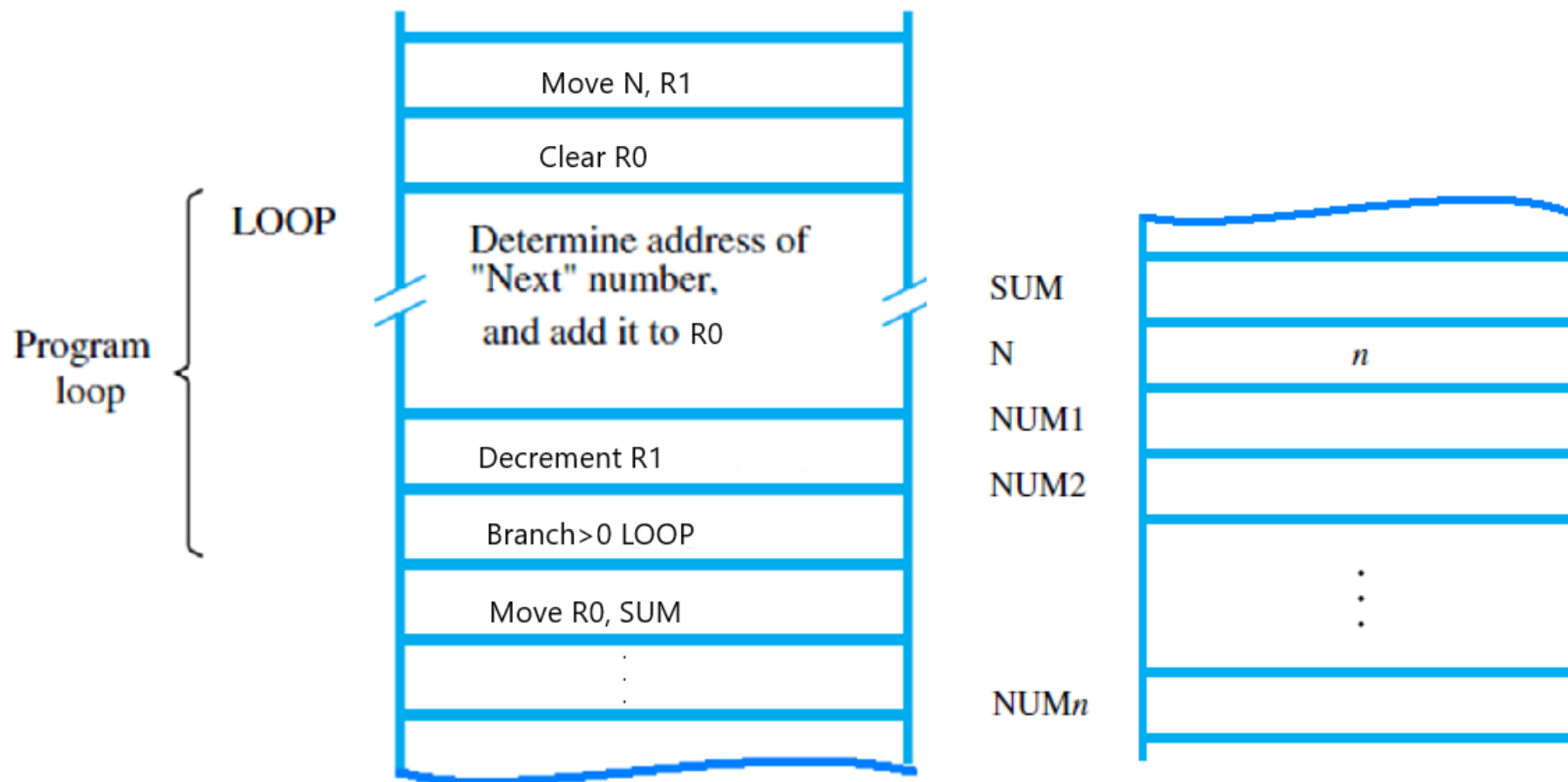
Branching



Branching

- Instead of using a long list of **Add** instructions, as in Figure, it is possible to implement a **program loop** in which the instructions read the next number in the list and add it to the current sum.
- To add all numbers, the loop has to be executed as many times as there are numbers in the list.
- The body of the loop is a straight-line sequence of instructions executed repeatedly. It starts at location **LOOP** and ends at the instruction **Branch>0 LOOP**.
- During each pass through this loop, the address of the next list entry is determined, and that entry is fetched and added to **R0**.

Branching



Branching

- For now, we concentrate on how to create and control a program loop.
- Assume that the number of entries in the list, ***n***, is stored in memory location **N**, as shown.
- Register **R1** is used as a counter to determine the number of times the loop is executed.
- Hence, the contents of location **N** are loaded into register **R1** at the beginning of the program.
- Then, within the body of the loop, the instruction

Decrement R1

reduces the contents of **R1** by **1** each time through the loop.

- Execution of the loop is repeated as long as the contents of **R1** are greater than zero.

Branching

We now introduce branch instructions.

- This type of instruction loads a new address into the program counter.
- As a result, the processor fetches and executes the instruction at this new address, called the branch target, instead of the instruction at the location that follows the branch instruction in sequential address order.
- A conditional branch instruction causes a branch only if a specified condition is satisfied.
- If the condition is not satisfied, the **PC** is incremented in the normal way, and the next instruction in sequential address order is fetched and executed.
- In the program in Figure, the instruction

Branch>0 LOOP

is a conditional branch instruction that causes a branch to location **LOOP** if the contents of register **R1** are greater than **zero**.

Branching

- This means that the loop is repeated as long as there are entries in the list that are yet to be added to **R0**.
- At the end of the *n*-th pass through the loop, the **Decrement** instruction produces a value of **zero** in **R1**, and, hence, branching does not occur.
- Instead, the **Move** instruction is fetched and executed. It moves the final result from **R0** into memory location **SUM**.

Condition codes

- The processor keeps track of information about the results of various operations for use by subsequent conditional branch instructions.
- This information is recorded in individual bits, called, *conditional code flags*
- these flags are grouped together in a special processor register called ***status register or conditional code register***

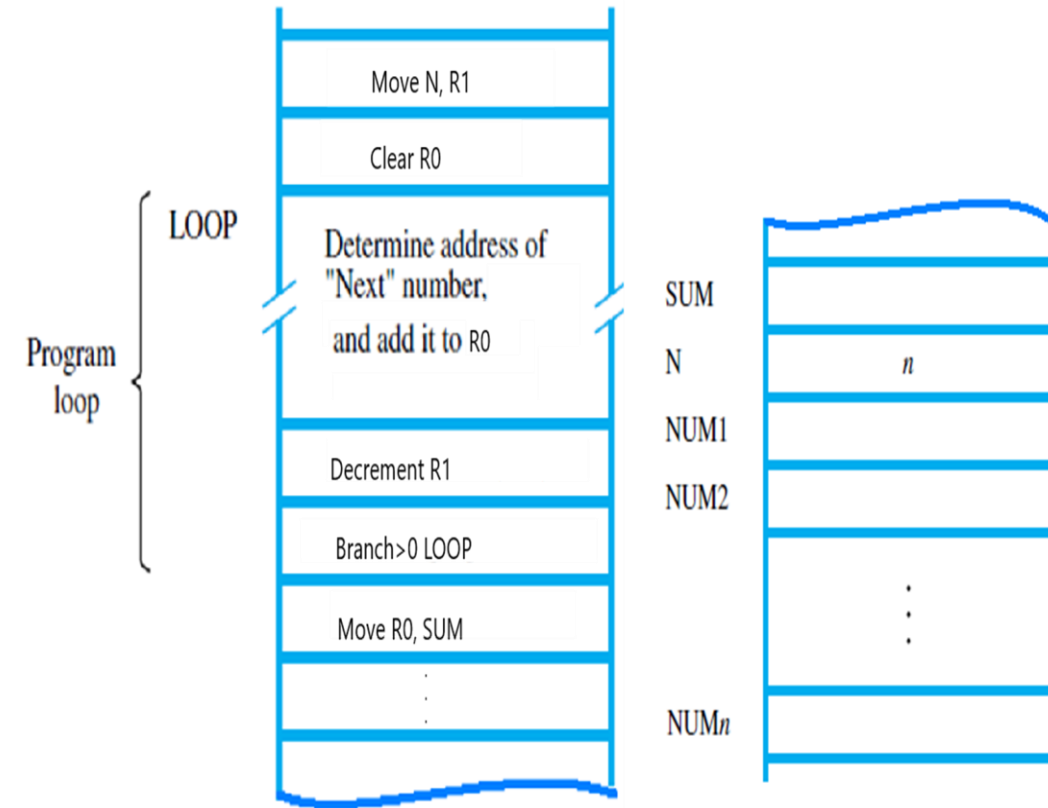
- | Flag | Use |
|--------------|---|
| N (negative) | Set to 1 if the result is negative; otherwise, cleared to 0 |
| Z (zero) | Set to 1 if the result is 0; otherwise, cleared to 0 |
| V (overflow) | Set to 1 if arithmetic overflow occurs; otherwise, cleared to 0 |
| C (carry) | Set to 1 if a carry-out results from the operation; otherwise, cleared to 0 |

Addressing Modes

Generating Memory Addresses

Let us return to example.

- The purpose of the instruction block starting at **LOOP** is to add successive numbers from the list during each pass through the loop.
- Hence, the **Load** instruction in that block must refer to a different address during each pass.
- How are the addresses specified?
 - The memory operand address cannot be given directly in a single **Load** instruction in the loop.
 - Otherwise, it would need to be modified on each pass through the loop.
 - As one possibility, suppose that a processor register, **R_i**, is used to hold the memory address of an operand.
 - If it is initially loaded with the address **NUM1** before the loop is entered and is then incremented by 4 on each pass through the loop, it can provide the needed capability.



Addressing Modes

- In general, a program operates on data that reside in the computer's memory.
- These data can be organized in a variety of ways that reflect the nature of the information and how it is used.
- Programmers use data structures such as lists and arrays for organizing the data used in computations.
- Programs are normally written in a high-level language, which enables the programmer to conveniently describe the operations to be performed on various data structures.
- When translating a high-level language program into assembly language, the compiler generates appropriate sequences of low-level instructions that implement the desired operations.
- *The different ways for specifying the locations of instruction operands are known as **addressing modes**.*

Addressing Modes

Name	Assembler Syntax	Addressing function
Immediate	#value	Operand = Value
Register	Ri	EA = Ri
Absolute (Direct)	LOC	EA = LOC
Indirect	(Ri) (LOC)	EA = [Ri] EA = (LOC)
Index	X(Ri)	EA = [Ri] + X
Base with Index	(Ri, Rj)	EA = [Ri] + [Rj]
Base with index and offset	X(Ri, Rj)	EA = [Ri] + [Rj] + X
Relative	X(PC)	EA = [PC] + X
Auto increment	(Ri)+	EA = [Ri]; increment Ri
Auto decrement	(Ri)-	Decrement Ri; EA = [Ri]
EA = Effective Address Value = a signed number X = Index value		

A summary on addressing modes.

Implementation of Variables and Constants

- In assembly language, a variable is represented by allocating a register or a memory location to hold its value.
- This value can be changed as needed using appropriate instructions.
- The program analyzed uses only two addressing modes to access variables.
- We access an operand by specifying the name of the register or the address of the memory location where the operand is located. The precise definitions of these two modes are:

Register mode—The operand is the contents of a processor register; the name of the register is given in the instruction.

Absolute mode—The operand is in a memory location; the address of this location is given explicitly in the instruction.

Implementation of Variables and Constants

- The instruction

Add R2, R3, R4

uses the **Register mode** for all three operands. Registers **R2** and **R3** hold the two source operands, while **R4** is the destination.

- The **Absolute mode** can represent global variables in a program. A declaration such as
int NUM1, NUM2, SUM;
- in a high-level language program will cause the compiler to allocate a memory location to each of the variables **NUM1**, **NUM2**, and **SUM**.
- Whenever they are referenced later in the program, the compiler can generate assembly-language instructions that use the **Absolute mode** to access these variables.
- The **Absolute mode** is used in the instruction

Load NUM1, R2 or Move LOC, R2

which loads the value in the memory location **NUM1/LOC** into register **R2**.

Implementation of Variables and Constants

- Constants representing data or addresses are also found in almost every computer program. Such constants can be represented in assembly language using the Immediate addressing mode. HLL has a statement like $A = B + 6$
- **Immediate mode**—The operand is given explicitly in the instruction.
- For example, the instruction

Move #200, R0

Places the value 200 in register **R0**.

Move B, R1

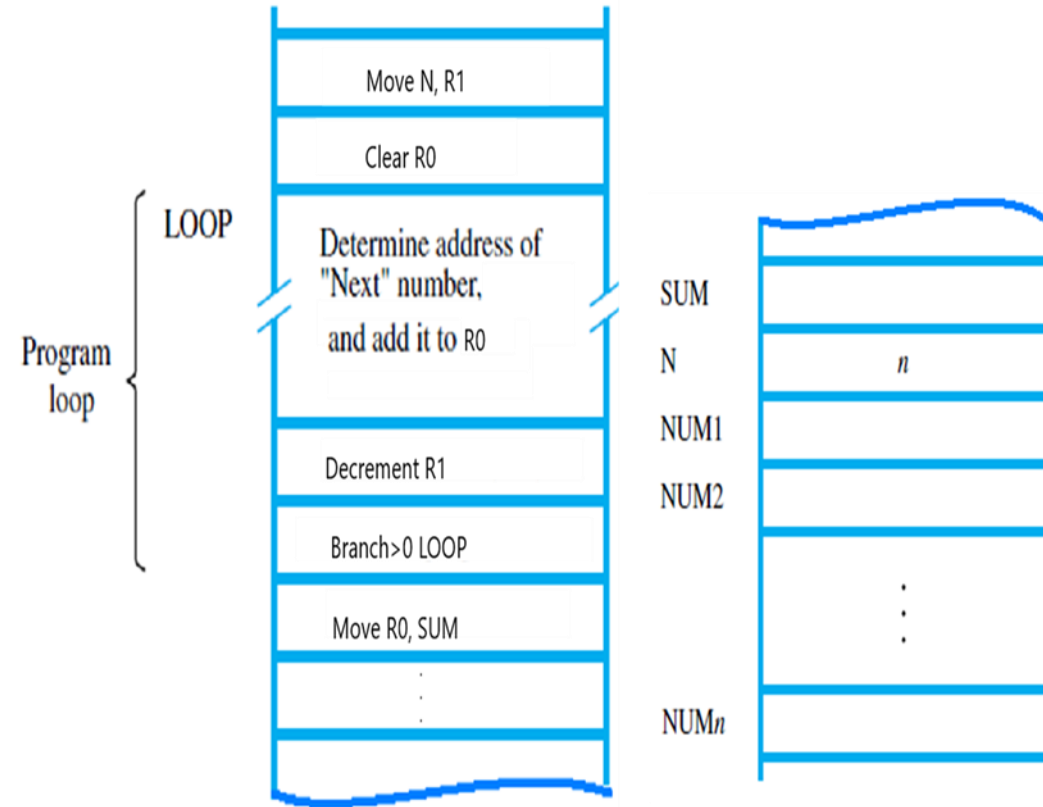
Add #6, R1

Move R1, A

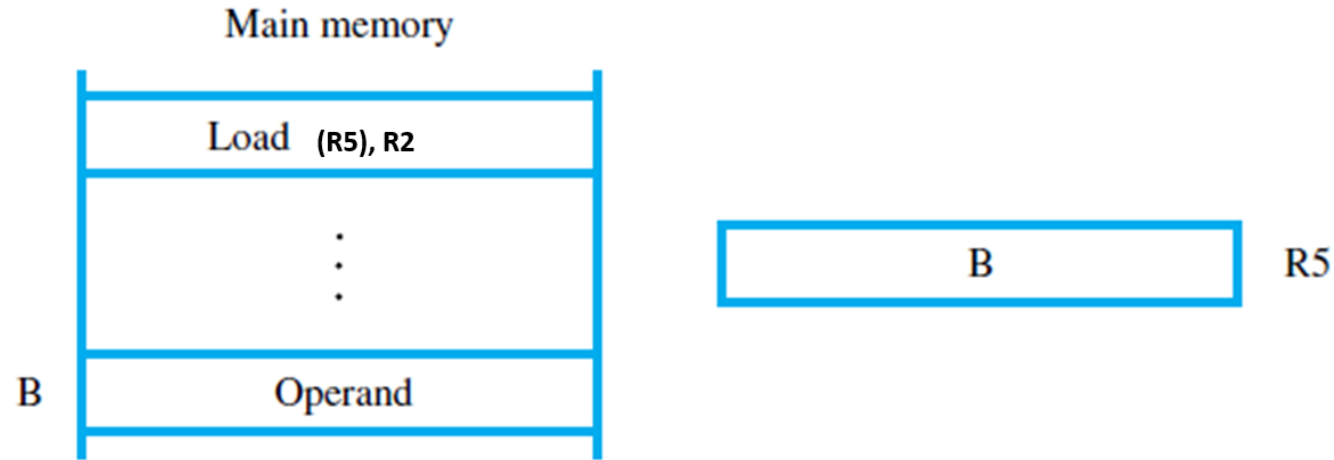
- A common convention is to use the number sign (#) in front of the value to indicate that this value is to be used as an immediate operand.
- In the addressing modes that follow, the instruction does not give the operand or its address explicitly.
- Instead, it provides information from which an **effective address (EA)** can be derived by the processor when the instruction is executed. The effective address is then used to access the operand.

Indirection and Pointers

- The program in last example requires a capability for modifying the address of the memory operand during each pass through the loop.
- A good way to provide this capability is to use a processor register to hold the address of the operand.
- The contents of the register are then changed (incremented) during each pass to provide the address of the next number in the list that has to be accessed.
- The register acts as a pointer to the list, and we say that an item in the list is accessed indirectly by using the address in the register.
- The desired capability is provided by the **indirect addressing mode**.



Indirection and Pointers

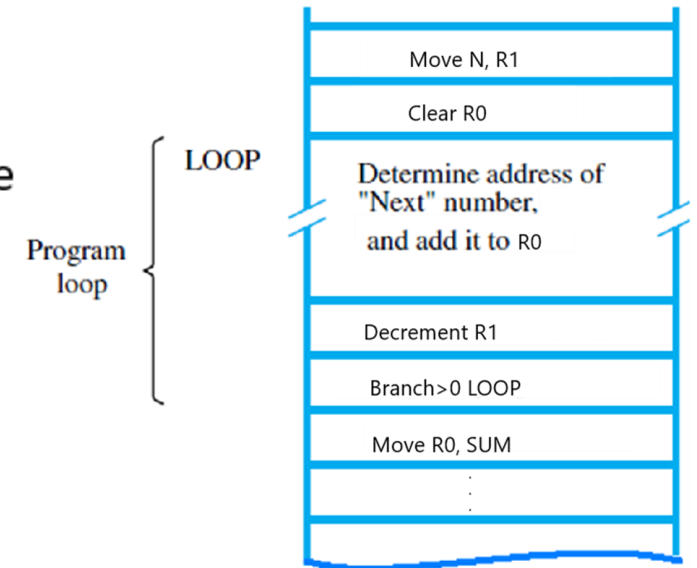


- **Indirect mode**—The effective address of the operand is the contents of a register that is specified in the instruction.
- We denote indirection by placing the name of the register given in the instruction in parentheses as illustrated in Figure.
- To execute the Load instruction in Figure, the processor uses the value B, which is in register R5, as the effective address of the operand. It requests a Read operation to fetch the contents of location B in the memory.
- The value from the memory is the desired operand, which the processor loads into register R2.

Indirection and Pointers

- Let us now return to the program with loop for adding a list of numbers.
- **Indirect addressing** can be used to access successive numbers in the list, resulting in the program shown here:

	Move	N, R1	Load the size of the List
	Move	#NUM1, R2	Get address of the first number
	Clear	R0	Initialize Sum to 0
LOOP	Add	(R2), R0	Add current number to Sum
	Add	#4, R2	Increment pointer to the list
	Decrement	R1	Decrement the counter by 1
	Branch>0	LOOP	Branch back if not finished
	Move	R0, SUM	Store the final Sum



Indirection and Pointers

- As another example of pointers, consider the C-language statement

A = *B;

- where **B** is a pointer variable and the ‘*****’ symbol is the operator for **indirect accesses**.
- This statement causes the contents of the memory location pointed to by **B** to be loaded into memory location **A**.
- The statement may be compiled into as follows

Move B, R2
Load (R2), R3
Store R3, A

Or

Move B, R2
Move (R2), A

Or

Move (B), A

- Indirect addressing through registers is used extensively.

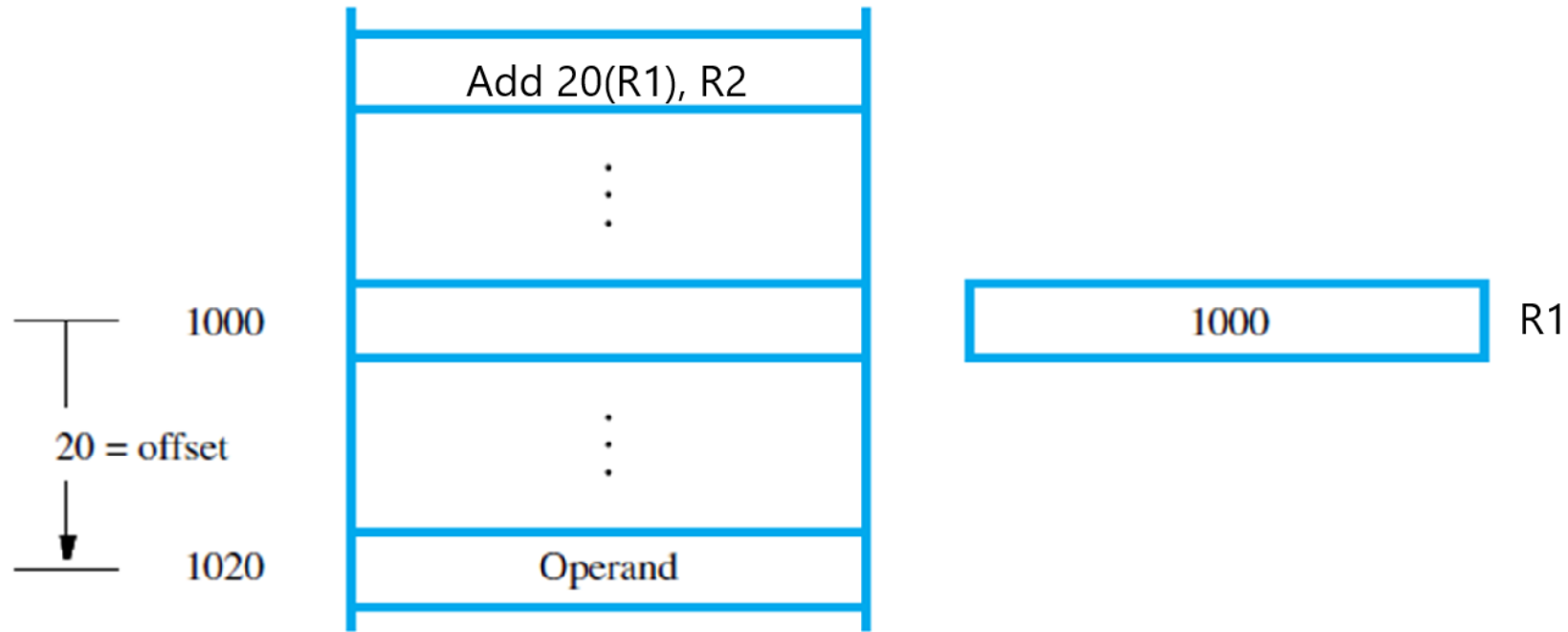
Indexing and Arrays

- The next addressing mode we discuss provides a different kind of flexibility for accessing operands. It is useful in dealing with lists and arrays.
- **Index mode**—The effective address of the operand is generated by adding a constant value to the contents of a register.
- For convenience, we will refer to the register used in this mode as the index register.
- Typically, this is just a general-purpose register. We indicate the Index mode symbolically as

$$\mathbf{X(Ri)}$$

- where **X** denotes a constant signed integer value contained in the instruction and **Ri** is the name of the register involved.
- The effective address of the operand is given by **EA = X + [Ri]**
- The contents of the register are not changed in the process of generating the effective address.

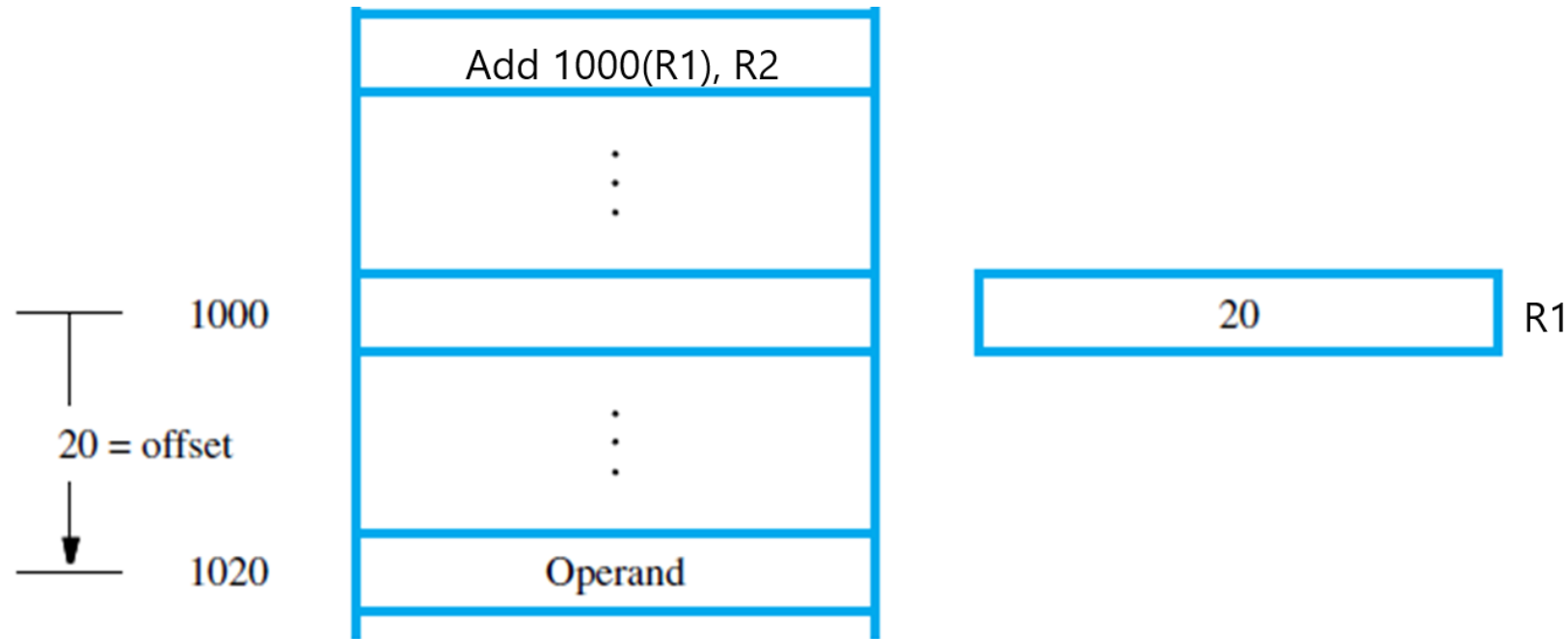
Indexing and Arrays



(a) Offset is given as a constant

The index register **R1** contains the address of a memory location, and the value **x** defines an **offset** (also called a **displacement**) from this address to the location where the operand is found.

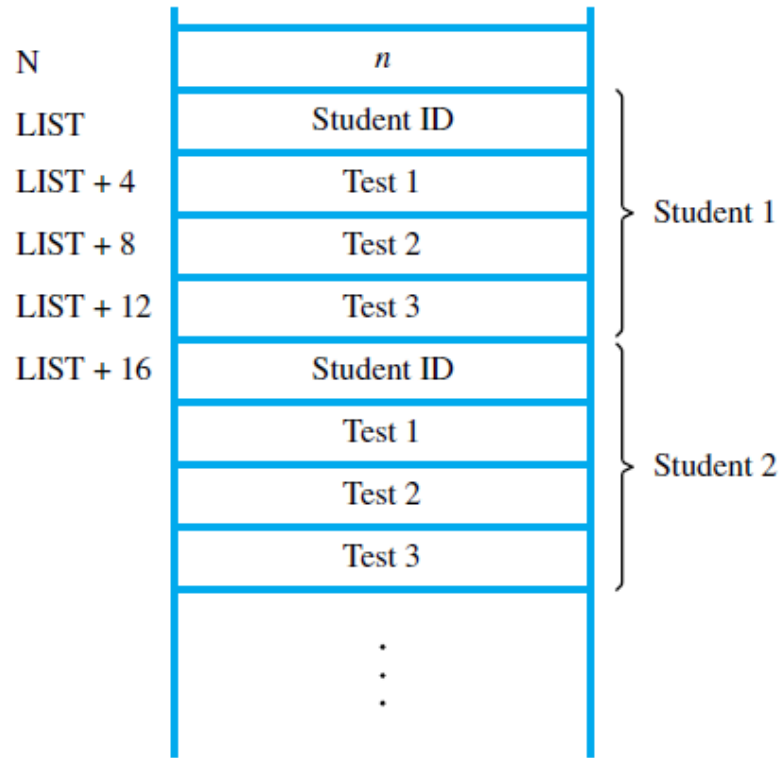
Indexing and Arrays



(b) Offset is in the index register

- The constant X corresponds to a memory address, and the contents of the index register define the offset to the operand.
- In either case, the effective address is the sum of two values; one is given explicitly in the instruction, and the other is held in a register.

Indexing and Arrays

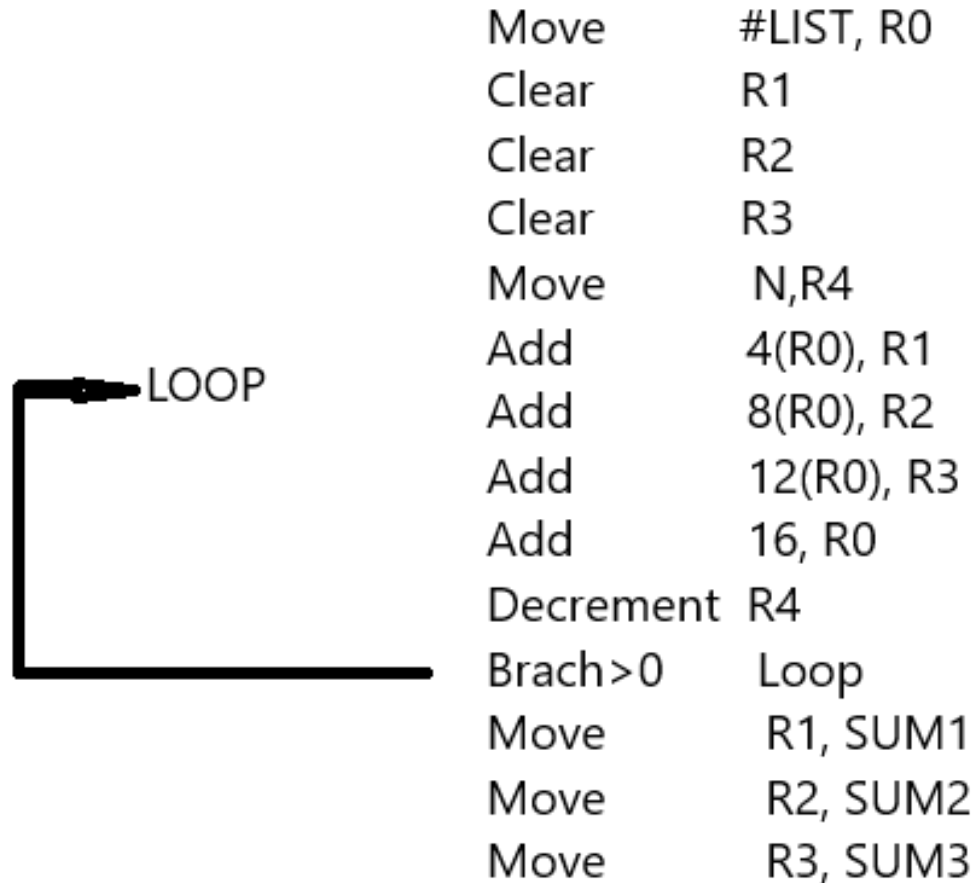


- Assume that the list of scores, beginning at location **LIST**, is structured as shown.
- A four-word memory block comprises a record that stores the relevant information for each student.
- Each record consists of the student's identification number (ID), followed by the scores the student earned on three tests.
- There are n students in the class, and the value n is stored in location **N** immediately in front of the list.
- The addresses given in the figure for the student IDs and test scores assume that the memory is byte addressable and that the word length is 32 bits.

Indexing and Arrays

- We should note that the list represents a two-dimensional array having n rows and four columns.
- Each row contains the entries for one student, and the columns give the IDs and test scores.
- Suppose that we wish to compute the sum of all scores obtained on each of the tests and store these three sums in memory locations **SUM1**, **SUM2**, and **SUM3**.
- In the body of the loop, the program uses the **Index addressing mode** in the manner depicted in Figure to access each of the three scores in a student's record.
- Register **R0** is used as the index register. Before the loop is entered, **R0** is set to point to the ID location of the first student record which is the address **LIST**.

Indexing and Arrays



Indexing addressing used in accessing test scores

Indexing and Arrays

- On the first pass through the loop, test scores of the first student are added to the running sums held in registers **R1**, **R2**, and **R3**, which are initially cleared to **0**.
- These scores are accessed using the **Index addressing modes 4(R0), 8(R0), and 12(R0)**.
- The index register **R0** is then incremented by 16 to point to the ID location of the second student.
- Register **R4**, initialized to contain the value n , is decremented by 1 at the end of each pass through the loop.
- When the contents of **R4** reach 0, all student records have been accessed, and the loop terminates.
- Until then, the conditional branch instruction transfers control back to the start of the loop to process the next record.
- The last three instructions transfer the accumulated sums from registers **R1**, **R2**, and **R3**, into memory locations **SUM1**, **SUM2**, and **SUM3**, respectively.

Indexing and Arrays

- Several variations of this basic form provide for efficient access to memory operands in practical programming situations (although they may not be included in some processors).
- For example, a second register **R_j** may be used to contain the offset **X**, in which case we can write the Index mode as

(R_i,R_j)

- The effective address is the sum of the contents of registers **R_i** and **R_j**.
- The second register is usually called the **base** register. This form of indexed addressing provides more flexibility in accessing operands, because both components of the effective address can be changed.
- Yet another version of the Index mode uses two registers plus a constant, which can be denoted as

X(R_i,R_j)

- In this case, the effective address is the sum of the constant **X** and the contents of registers **R_i** and **R_j**.

Relative addressing

- **Relative mode:** The effective address is determined by the Index mode using the program counter in place of the general purpose register R_i
- $X(PC)$ is used to address a memory location that is X bytes away from the location presently pointed to by the PC.
- Since the addressed location is relative to the program counter hence the name.
- **Example:** Assume the 4 instructions of the loop body, starting at LOOP are located at memory locations 1000, 1004, 1008 and 1012.
 - Hence the updated contents of the PC at the time the branch target address is generated will be 1016.
 - To branch to location LOOP(1000), the offset value needed is $X = -16$.
 - The corresponding machine instruction generated using the addressing mode $-16(PC)$



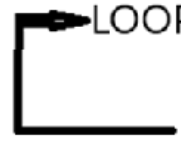
```
Move      N, R1
Move      #NUM1, R2
Clear     R0
LOOP Add   (R2), R0
Add       #4, R2
Decrement R1
Branch > 0 LOOP
Move      R0, SUM
```

Additional Modes

- Additional modes intended to aid certain programming tasks.
- **Auto-increment mode** – the effective address of the operand is the contents of a register specified in the instruction. After accessing the operand, the contents of this register are automatically incremented to point to the next item in the list.
- It is written as $(R_i)^+$
- Implicitly the increment is 1.
- Computers that have the autoincrement mode automatically increment the contents of the register by a value that correspond to the size of the accessed operand.
- Thus, the increment is 1 for byte-sized operand, 2 for 16-bit operands and 4 for 32-bit operands.



```
Move    N, R1
Move    #NUM1, R2
Clear   R0
LOOP:   Add    (R2), R0
        Add    #4, R2
        Decrement R1
        Branch>0 LOOP
        Move    R0, SUM
```



```
Move    N, R1
Move    #NUM1, R2
Clear   R0
LOOP:   Add    (R2) +, R0
        Decrement R1
        Branch>0 LOOP
        Move    R0, SUM
```

Additional Modes

- **Auto-decrement mode:** The contents of a register specified in the instruction are ***first automatically decremented*** and are then used as the effective address of the operand.
- It is written as $-(R_i)$
- The operands are accessed in descending address order
- Used in stack operations
- Autoincrement and autodecrement addressing modes combines two instructions into one instruction. One to access operand and other to increment or decrement respectively the register that contains the operand address.
- It reduces the number of instructions needed to perform the desired task.

Assembly Language

- We use symbolic names (like LOAD, MOV, ADD, INC) to represent machine instructions as representing them by patterns of 0s and 1s is awkward.
- LOAD, MOV, ADD, INC are called **mnemonics**.
- A complete set of such symbolic names and rules for their use constitutes a programming language called an assembly language.
- The set of rules for using the mnemonics in the specification of complete instructions and programs called the **syntax** of the language.
- Programs written in an assembly language are translated into a sequence of machine instruction by a program called an **assembler**.
- Assembler is a part of the system software.
- The user program in its original alphanumeric text format is called a source program and the assembled machine program is called object program.
- Example: MOVE R0, SUM
- MOVE represents the binary pattern, called **OP code** for the operation performed by the instruction.
- The assembler translates the mnemonic into the binary OP code

Assembler Directives

- Names are used to assign numerical values
 - Example: SUM EQU 200
- This statement doesn't denote an instruction that will be executed when the assembly program run.
- This statement informs the assembler the name SUM should be replaced by the value 200 wherever it appears in the program.
- These statements which are directives to assembler are called assembler directives (or commands)

	100	Move	N,R1
	104	Move	#NUM1,R2
	108	Clear	R0
LOOP	112	Add	(R2),R0
	116	Add	#4,R2
	120	Decrement	R1
	124	Branch>0	LOOP
	128	Move	R0,SUM
	132		
			:
SUM	200		
N	204		100
NUM1	208		
NUM2	212		
			:
NUMn	604		

Memory arrangement for the program

If the assembler is to produce an object Program according to this arrangement, it has to know:

- How to interpret the names
- Where to place the instructions in the memory
- Where to place the data operands in the memory
- EQU means variable is assigned to value
- ORIGIN informs the assembler where in the memory to place the data block that follows
- DATAWORD informs the assembler the data value to be placed in the memory word at address ORIGIN
- RESERVE informs the assembler the size of the memory block reserved for data
- END tells the assembler it is the end of the source program text
- START – address of the location at which execution of the program to begin

	Memory address label	Operation	Addressing or data information	Where to place data block
Assemblerdirectives	SUM	EQU	200	
		ORIGIN	204	
	N	DATAWORD	100	First data
	NUM1	RESERVE	400	Reserve 400 bytes
Statements that generate machine instructions		ORIGIN	100	
	START	MOVE	N,R1	Starting address
		MOVE	#NUM1,R2	
		CLR	R0	
	LOOP	ADD	(R2),R0	
		ADD	#4,R2	
		DEC	R1	
		BGTZ	LOOP	
Assemblerdirectives		MOVE	R0,SUM	
		RETURN		
		END	START	

Assembly language representation for the program

- RETURN informs the assembler the point at which the execution of the program to be terminated

Assembler source program

- Assembly language statements format
 - Label Operation Operand(s) Comment
- Branch offset
 - In some cases the assembler does not directly replace a name representing an address with the actual value of the address
 - For example: The assembler computes the branch offset which is the distance to the target, and puts it into the machine instruction
- Symbol Table: as the assembler scans through a source program, it keeps track of all names and the numerical values that corresponds to them in a symbol table
 - When a name appears second time, it is replaced with its value from the table
 - When symbol name appears as an operand before it is given a value – it is a problem. Hence, two pass assemblers are developed

ThankQ